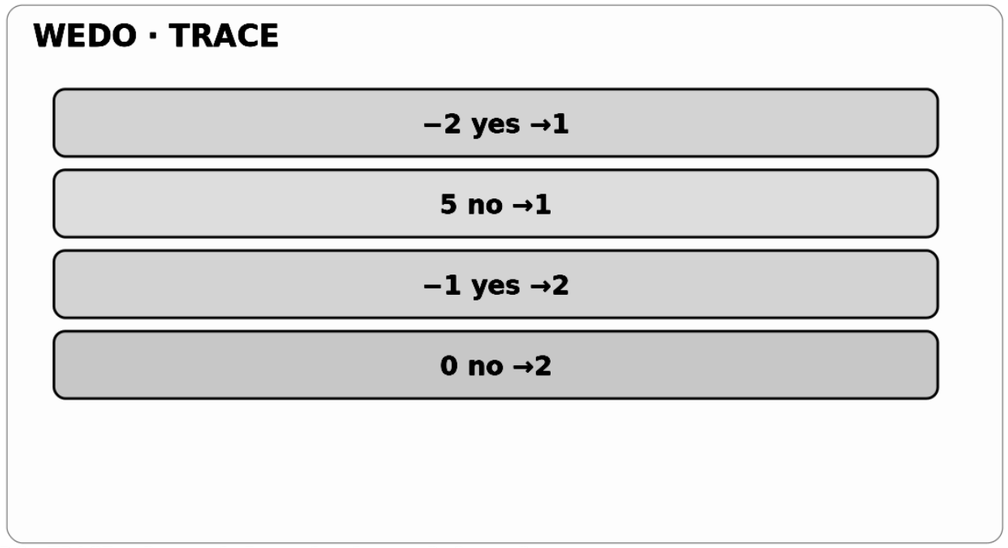
## We Do source CHECK · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Design a function counting temperatures below zero in [−2,5,−1,0].**

Zero does not count as negative, so use <0, not <=0.




WEDO · TRACE: −2 yes →1; 5 no →1; −1 yes →2; 0 no →2





**DO NOW — RECALL**

## Lesson 072 · Algorithm Design: From Specification to St

Name: _____ Class: _____ Date: _____

**1. What is a loop invariant?**

**2. What is a postcondition?**

**3. How is an input specification useful?**

**4. C# array a=[3,5,7]: set a[1]=a[0]+a[2]. Give final array and sum.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

**1. Model answer**

Property true after processing each step.

**2. Model answer**

Expected outcome after valid execution.

**3. Model answer**

Defines valid types, conditions and outputs.

**4. Model answer**

[3,10,7], sum20.

**By the end, you can…**

• Read the problem to identify exact input type, required output, boundaries and invalid cases.

• Write an algorithm with an invariant: after each iteration, count is the number of qualifying values examined so far.

• Choose meaningful tests around the 40 boundary and compare output against an independently calculated result.

**Vocabulary — know these terms**

algorithm — Finite unambiguous sequence of steps.

invariant — Property preserved across loop iterations.

precondition — Required input assumption.

postcondition — Expected condition after executing.





WE DO — CHECK AND IMPROVE

# Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

**Guided 1 — CHECK**

Explain
Zero does not count as negative, so use <0, not <=0.

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking.

**An algorithm can be evaluated without considering its input requirements.**

**Circle: Confident / Getting there / Need help**

Explain what you can now do independently.

**GREEN PEN — Correct and explain**

Choose one error. Write the correct method and explain precisely why your first response was wrong.





NEW LEARNING

# Read, annotate and explain

## Observe

Read the problem to identify exact input type, required output, boundaries and invalid cases.

## Explain

Write an algorithm with an invariant: after each iteration, count is the number of qualifying values examined so far.

## Evaluate

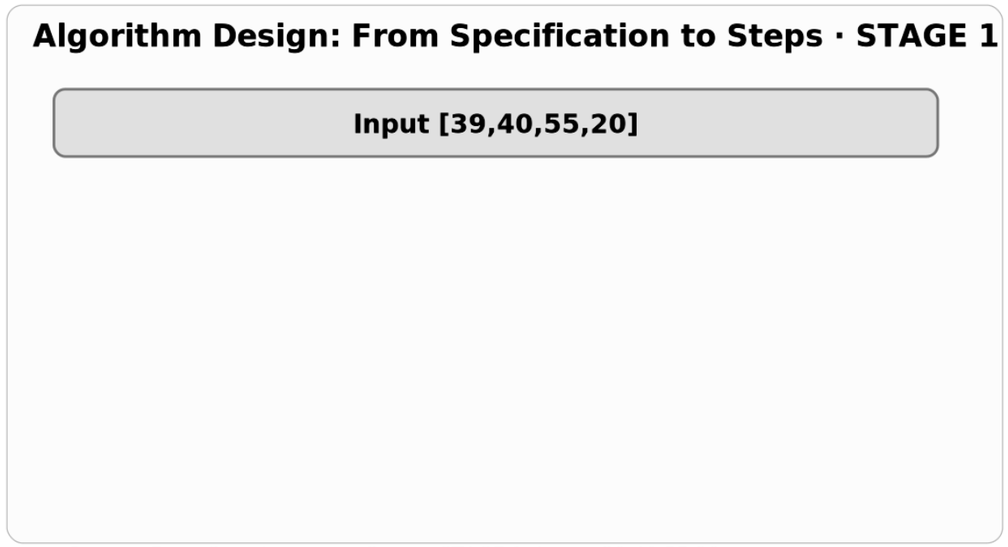
Choose meaningful tests around the 40 boundary and compare output against an independently calculated result.

**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**




Algorithm Design: From Specification to Steps · STAGE 1: Input [39,40,55,20]





**YOU DO — INDEPENDENT PRACTICE**

# Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

**Question 3**

Explain a loop invariant, time complexity and edge tests for CountPasses. [6]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

**Question 1**

Trace CountPasses for [40,41,39,100]. Show count after each item. [6]

**Question 2**

Write C# method that counts negative integers in an array, and returns0 for empty. [6]

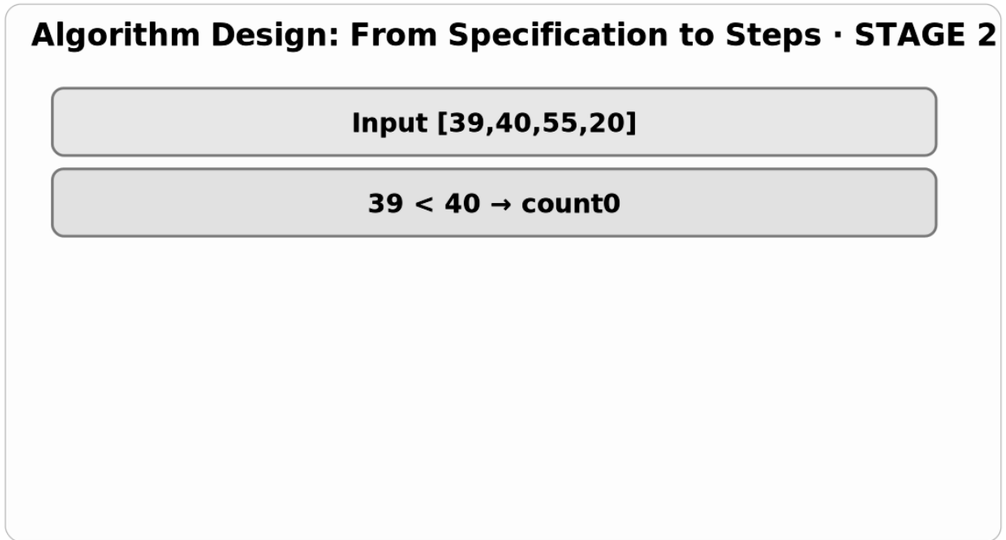




# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**

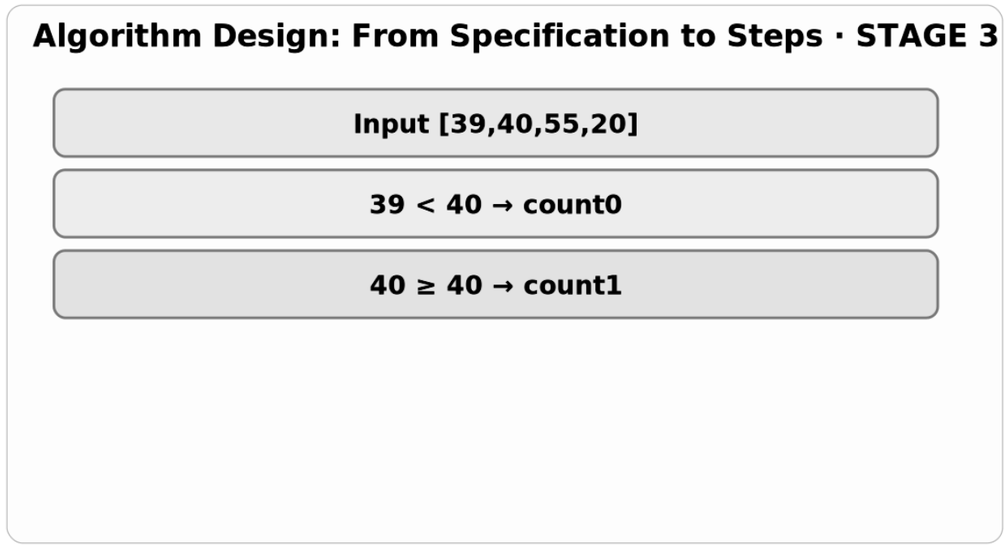



Algorithm Design: From Specification to Steps · STAGE 2: Input [39,40,55,20]; 39 < 40 → count0

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**




Algorithm Design: From Specification to Steps · STAGE 3: Input [39,40,55,20]; 39 < 40 → count0; 40 ≥ 40 → count1





**WE DO — GUIDED PRACTICE**

# Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

**Guided 1 — TRY**

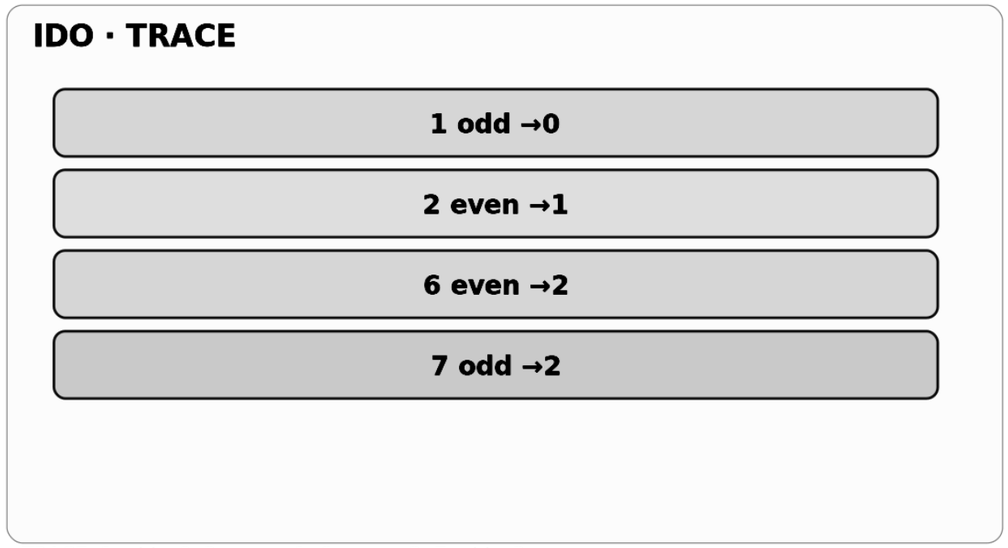
Design a function counting temperatures below zero in [−2,5,−1,0].
Source code / input:
If x<0 increment count; −2 and −1 qualify → result2.

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Design a function counting even numbers in [1,2,6,7].**

Using modulo2==0 identifies even values; the list has 2 and 6.




IDO · TRACE: 1 odd →0; 2 even →1; 6 even →2; 7 odd →2

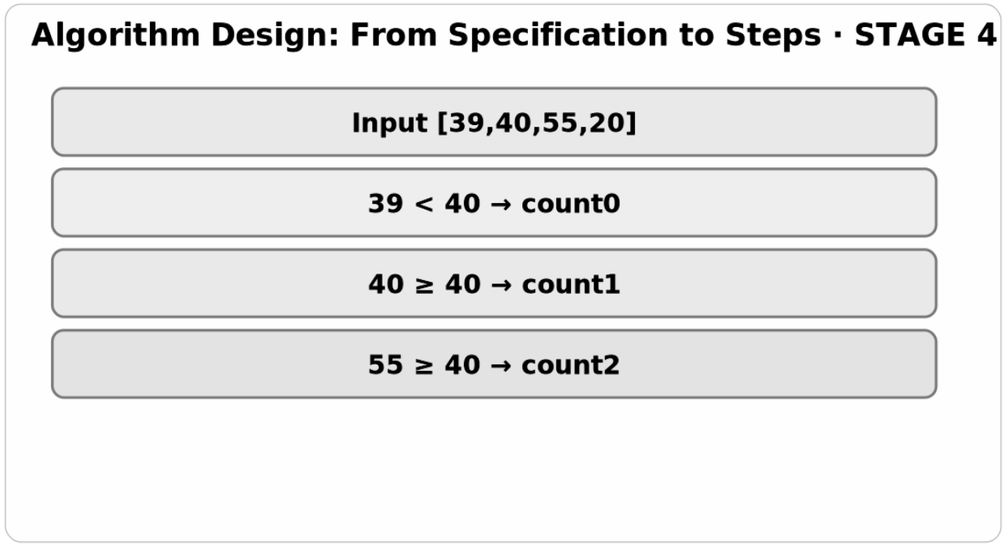




# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**

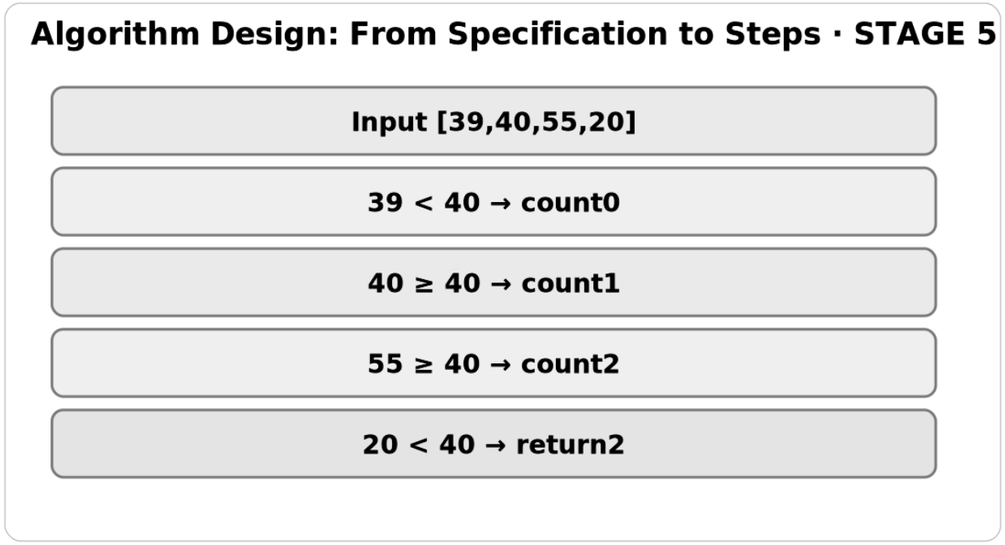



Algorithm Design: From Specification to Steps · STAGE 4: Input [39,40,55,20]; 39 < 40 → count0; 40 ≥ 40 → count1; 55 ≥ 40 → count2

## New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**




Algorithm Design: From Specification to Steps · STAGE 5: Input [39,40,55,20]; 39 < 40 → count0; 40 ≥ 40 → count1; 55 ≥ 40 → count2; 20 < 40 → return2





**I DO — WORKED EXAMPLE**

## Follow the worked method

**Design a function counting even numbers in [1,2,6,7].**

**Explain**

Using modulo2==0 identifies even values; the list has 2 and 6.